SDG BLOCKCHAIN ACCELERATOR

Debugging and Testing Report

1. Project Information

- **Project Name:** SHELTER
- **Challenge & UNDP Office:** MALAWI UNDP
- **Report Version:** 1.0

2. Testing Approach

LTW SERVICE:

To ensure the reliability of the off-chain application layer, we implemented a comprehensive unit and integration testing strategy using the Jest framework. The testing suite focuses on validating the core business logic, transaction orchestration, and API input handling.

All files

71.35% Statements `593/831` **54%** Branches `108/200` **78.13%** Functions `318/407` **72.62%** Lines `589/811`

Press *n* or *j* to go to the next uncovered block, *b*, *p* or *k* for the previous block.

Filter:

File	Statements		Branches		Functions		Lines	
beneficiaries/models/amount-of	100%	7/7	100%	1/1	100%	3/3	100%	7/7
beneficiaries/models/beneficiary-to-json	100%	2/2	100%	0/0	100%	2/2	100%	2/2
beneficiaries/models/beneficiary/null-beneficiary	100%	4/4	100%	0/0	100%	4/4	100%	4/4
beneficiaries/models/blockfrost-provider-service	100%	2/2	100%	1/1	100%	2/2	100%	2/2
beneficiaries/models/data-repo/phone-number-updates-data-repo/default	100%	4/4	100%	1/1	100%	4/4	100%	4/4
beneficiaries/models/data-repo/phone-number-updates-data-repo/fake	100%	4/4	100%	1/1	100%	4/4	100%	4/4
beneficiaries/models/messages/wallet-creation-msg	100%	2/2	100%	0/0	100%	2/2	100%	2/2
beneficiaries/models/otp	100%	3/3	100%	1/1	100%	1/1	100%	3/3
beneficiaries/models/phone-number-update/default-phone-number-update	100%	4/4	100%	0/0	100%	4/4	100%	4/4
beneficiaries/models/phone-number-update/null-phone-number-update	100%	3/3	100%	0/0	100%	3/3	100%	3/3
beneficiaries/models/script	100%	4/4	100%	2/2	100%	3/3	100%	4/4
beneficiaries/models/ttl	100%	5/5	100%	2/2	100%	2/2	100%	5/5
beneficiaries/serializers	100%	1/1	100%	0/0	100%	0/0	100%	1/1
db/mongo	100%	5/5	100%	0/0	100%	0/0	100%	5/5
fixtures	100%	11/11	100%	3/3	100%	2/2	100%	11/11

CARDANO:

Actors:

- **ADMIN / NGO (Steward):** The entity responsible for managing and locking the humanitarian aid (AID) into the smart contract. Their primary role is to initialize the voucher with the correct beneficiary data and recover funds if they are not used after the expiration period.

- **Beneficiary:** The end-user or recipient of the humanitarian aid. This actor is authorized to claim and spend the locked funds within a specific timeframe (claim_start to claim_end) to acquire essential goods or services.

US-01: Assign AID
As an ADMIN/NGO, **I want to** lock funds into the smart contract **so that** they are assigned to a specific beneficiary for humanitarian aid.
Acceptance Criteria: ● The transaction must be signed by the Steward to authorize the locking of funds. ● A valid VoucherDatum must be attached to the UTxO, defining both the participants and the valid claim period. ● The system must reject any attempt to create a voucher with missing or invalid metadata.

Test case:

US-01-TC-1:

- **Description:** Verify that the transaction is successful when the Steward signs the creation of the UTxO with a valid datum.
- **Expected Result:** Funds are correctly locked in the smart contract address.
- **Actual Result:** Funds were successfully locked as expected.✅

US-01-TC-2:

- **Description:** Verify that the smart contract rejects the creation of the AID if the datum containing the Beneficiary and Steward hashes, along with the validity timeframe, is not included.
- **Expected Result:** The smart contract prevents the AID creation without the required datum.
- **Actual Result:** The transaction failed as expected when the datum was missing or malformed.✅

US-01-TC-3:

- **Preconditions:** Attempting to assign AID for a specific beneficiary.
- **Description:** Verify that the funds remain correctly locked using the specific parameters defined in the datum (beneficiary, steward, claim_start, claim_end).
- **Expected Result:** The parameters utilized in the on-chain datum match the intended configuration for the locked funds.
- **Actual Result:** Verified that the on-chain datum parameters match the provided input values.✅

US-02: Consume AID
As a Beneficiary, **I want to** consume my assigned AID **so that** I can acquire the goods or services I need.
Acceptance Criteria: • The transaction must fail if the current time is before claim_start or after claim_end. • The AID can only be consumed if the transaction is signed by the Beneficiary and the validity range is within [claim_start, claim_end]. • Once the AID expires, it must be available for recovery by the Steward.

Test case:

US-02-TC-1:

- **Description:** Verify that the transaction is successful when the Beneficiary signs the request and the validity range is within the [claim_start, claim_end] interval.
- **Expected Result:** The AID is successfully consumed, and the transaction is validated by the smart contract.
- **Actual Result:** The transaction was accepted as the signature and time range were valid.✅

US-02-TC-2:

- **Description:** Verify that the transaction fails if the Beneficiary's cryptographic signature is missing.
- **Expected Result:** The smart contract rejects the transaction for lack of authorization.
- **Actual Result:** The transaction failed with the trace: [Contract Error]: BeneficiaryAction, must be signed with beneficiary key.✅

US-02-TC-3:

- **Description:** Verify that the transaction fails if attempted before the claim_start timestamp.
- **Expected Result:** The contract blocks the consumption because the claim period hasn't started yet.
- **Actual Result:** The transaction failed with the trace: [Contract Error]: BeneficiaryAction, must be valid after claim_start.✅

US-02-TC-4:

- **Description:** Verify that the transaction fails if attempted after the claim_end expiration time.
- **Expected Result:** The contract blocks the consumption because the AID has expired.
- **Actual Result:** The transaction failed with the trace: [Contract Error]: BeneficiaryAction, must be valid before claim_end.✅

US-03: Recover AID expired
As an **ADMIN/NGO**, I want to recover expired AID funds so that capital does not remain permanently locked in the contract.
Acceptance Criteria: • The transaction must fail if attempted before or during the valid claim range (claim_start to claim_end). • The AID can only be recovered if the transaction is signed by the Steward and the validity range starts strictly after claim_end.

Test case:

US-03-TC-1:

- **Description:** Verify that the transaction is successful when the Steward signs the recovery and the validity range starts strictly after claim_end.
- **Expected Result:** The Steward successfully recovers the expired funds.
- **Actual Result:** The funds were released back to the Steward once the expiration date was met.✅

US-03-TC-2:

- **Description:** Verify that the transaction fails if the Steward attempts to recover the funds before the claim_end timestamp.
- **Expected Result:** The smart contract rejects the recovery as the claim period for the beneficiary is still active.
- **Actual Result:** The transaction failed with the trace: [Contract Error]: StewardAction, must be valid after claim_end.✅

US-03-TC-3:

- **Description:** Verify that the transaction fails if the Steward's cryptographic signature is missing.
- **Expected Result:** The smart contract blocks the unauthorized recovery attempt.

- **Actual Result:** The transaction failed with the trace: [Contract Error]: StewardAction, must be signed with steward key.✅

US-03-TC-4:

- **Description:** Verify that the transaction fails if a Beneficiary attempts to execute the StewardAction recovery.
- **Expected Result:** The contract rejects the action because it lacks the authorized Steward signature.
- **Actual Result:** The transaction failed as the Beneficiary is not authorized for this action.✅

- **Unit Testing:** Written using the aiken test command and the mocktail library. We used a TestCase pattern to simulate multiple transaction scenarios, focusing on the extra_signatories and validity_range fields**.**

- **Integration Testing:**
  - Integration tests conducted on Cardano Preview Testnet
  - Sample Transactions:
    - Action: Assign AID
      - Network: Cardano Preview
      - TxHash: f5cea9526cec021b74fc35fdec275ea24ddfb7221026f49c493c10d32da8298c
      - Explorer Link: https://preview.cardanoscan.io/transaction/f5cea9526cec021b74fc35fdec275ea24ddfb7221026f49c493c10d32da8298c
    - Action: Consume AID
      - Network: Cardano Preview
      - TxHash: be1b89805134ad076f2a0ddb45c681d900138679b30629dcdaaf008626b6fbbc
      - Explorer Link: https://preview.cardanoscan.io/transaction/be1b89805134ad076f2a0ddb45c681d900138679b30629dcdaaf008626b6fbbc
    - Action: Recover AID
      - Network: Cardano Preview
      - TxHash: 051680893a2e67106a5cd5cd7be3cbea58e05b31c27d677798c98c486b17bc9f

- Explorer Link: https://preview.cardanoscan.io/transaction/051680893a2e67106a5cd5cd7be3cbea58e05b31c27d677798c98c486b17bc9f

3. Error Logs

- **Error 1:** The transaction was rejected because the input UTxO did not contain the required VoucherDatum.

```
    │ PASS [mem:  42297, cpu: 17139239] datum_none
    │ · with traces
    │ │ [Contract Error]: datum not found in UTxO
```

- **Error 2:** A recovery attempt via StewardAction failed because the required cryptographic signature from the Steward was missing.

```
    │ PASS [mem:  86210, cpu: 36666904] steward_not_sign
    │ · with traces
    │ │ [Contract Error]: StewardAction, must be signed with steward key
```

- **Error 3:** The Steward attempted to recover funds before the expiration date (claim_end). The contract enforces that recovery is only possible after the claim period ends.

```
    │ PASS [mem: 118208, cpu: 48468846]
steward_not_recover_before_claim_end
    │ · with traces
    │ │ [Contract Error]: StewardAction, must be valid after claim_end
```

- **Error 4:** The Beneficiary attempted to execute the StewardAction. The transaction was blocked as it lacked the authorized Steward signature.

```
    │ PASS [mem: 107806, cpu: 44289656] beneficiary_not_recover
    │ · with traces
    │ │ [Contract Error]: StewardAction, must be signed with steward key
```

- **Error 5:** An attempt to consume the AID via BeneficiaryAction failed due to the absence of the Beneficiary's signature.

```
    │ PASS [mem: 116210, cpu: 47550099] beneficiary_not_sign
    │ · with traces
```

```
    │ │ [Contract Error]: BeneficiaryAction, must be signed with
beneficiary key
```

- **Error 6:** The Beneficiary tried to consume the AID before the allowed claim_start timestamp.

```
    │ PASS [mem:  91429, cpu: 38736072]
beneficiary_not_consume_before_claim_start
    │ · with traces
    │ │ [Contract Error]: BeneficiaryAction, must be valid after
claim_start
```

- **Error 7:** The Beneficiary tried to consume the AID after the claim_end expiration time.

```
    │  PASS [mem: 134629, cpu: 54646396]
beneficiary_not_consume_after_claim_end
    │ · with traces
    │ │ [Contract Error]: BeneficiaryAction, must be valid before claim_end
```

- **Error 8:** The Steward attempted to execute the BeneficiaryAction. The transaction was rejected because it requires the Beneficiary's signature, not the Steward's.

```
    │ PASS [mem: 137806, cpu: 55172851] steward_not_consume
    │ · with traces
    │ │ [Contract Error]: BeneficiaryAction, must be signed with
beneficiary key
```

4. Resolved Issues

Issue ID	Description	Root Cause	Resolution	Status
001	Ambiguous transaction logic separation.	Lack of a defined Redeemer type for different user actions.	Implemented a custom Redeemer type with StewardAction and BeneficiaryAction to clearly separate off-chain logic.	✅ Fixed

002	Potential runtime errors due to missing Datums.	The validator did not explicitly handle the Option type for incoming UTxO datums.	Refactored the script to use a when datum_opt is { Some -> ..., None -> ... } structure to safely handle missing data.	✅ Fixed
003	Opaque contract failures during debugging.	Validation conditions did not provide feedback upon failure.	Integrated explicit trace messages for each logic branch and adopted the -t verbose build flag for better error reporting.	✅ Fixed
004	Build TX invalidBefore & invalidHereAfter	The transaction builder (MeshJS) lacked the specific invalidBefore and invalidHereAfter properties required to satisfy the validator's validity range check.	Implemented explicit validity intervals in the transaction builder, setting a valid time window (current slot ± safety margin) to ensure the transaction falls within the allowed claim period.	✅ Fixed

5. Optimization Notes

The validator was developed following a correctness first approach. Instead of post-development micro-optimizations, the focus was on utilizing high-level abstractions that ensure predictable execution costs and maximum security.

Performance Highlights

- *Library Leveraging: Used vodka utility libraries for signature and validity range checks, which are pre-optimized for efficient UPLC (Untyped Plutus Core) generation.*
- *Logic Branching: Implemented a clean when (pattern matching) structure, which Aiken compiles into efficient execution paths, ensuring that only the necessary logic for each Redeemer is evaluated.*
- *Trace Management & Error Handling: Error messages are currently structured with trace for maximum debugging clarity during the development phase.*

For production, these if/else blocks combined with trace can be refactored into expect statements. This would further reduce the script size and execution cost by stripping string literals and simplifying the control flow while maintaining strict validation.

6. Tools and Environments Used

Blockchain & Smart Contracts

- Cardano Preview Testnet
- Aiken v1.1.17
- Vodka / Cocktail / Mocktail
- Cardano Node & CLI
- Blockfrost

Frontend Ecosystem (Merchant Platform)

- **Framework:** Next.js 16 (React 19)
- **UI & Styling:** Tailwind CSS v4 & Radix UI
- **Blockchain Integration:** Mesh SDK (@meshsdk/core)
- **Validation:** Zod & React Hook Form
- **Testing:** Vitest & Playwright

Backend Ecosystem (LTW Service)

- **Runtime:** Node.js (v20+) & TypeScript
- **Testing:** Jest
- **Blockchain Integration:** Mesh SDK (@meshsdk/core)
- **Integrations:** Twilio SDK & Axios
- **Database:** PostgreSQL & FerretDB
- **Infrastructure:** Docker

7. Remaining Issues / Next Steps

- **Production Refactoring** (Trace Removal): Currently, the validator uses verbose trace messages for debugging. The next step is to refactor these into expect statements to strip string literals, reducing script size and transaction fees.
- **LTW Service Optimization**: Refactor backend components to improve modularity and expand unit/integration test coverage (using Jest) to >80% coverage.
- **Merchant Platform Refactoring**: Implement automated component testing (using Vitest) to ensure UI reliability and seamless wallet interaction.
- **Preprod Stress Testing**: Move the testing environment from Preview to Preprod to simulate high-concurrency scenarios to ensure the system scales correctly.
- **Security Audit**: Conduct an external security audit focusing on the time-lock logic and backend key management.
- **Rahat Integration**: Research and test the feasibility of incorporating the Rahat Dashboard components to enhance the system's reporting capabilities and UI for non-technical NGO administrators.